ASSIGNMENT 12.1

Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

Konuganti Ruthwik Bhat

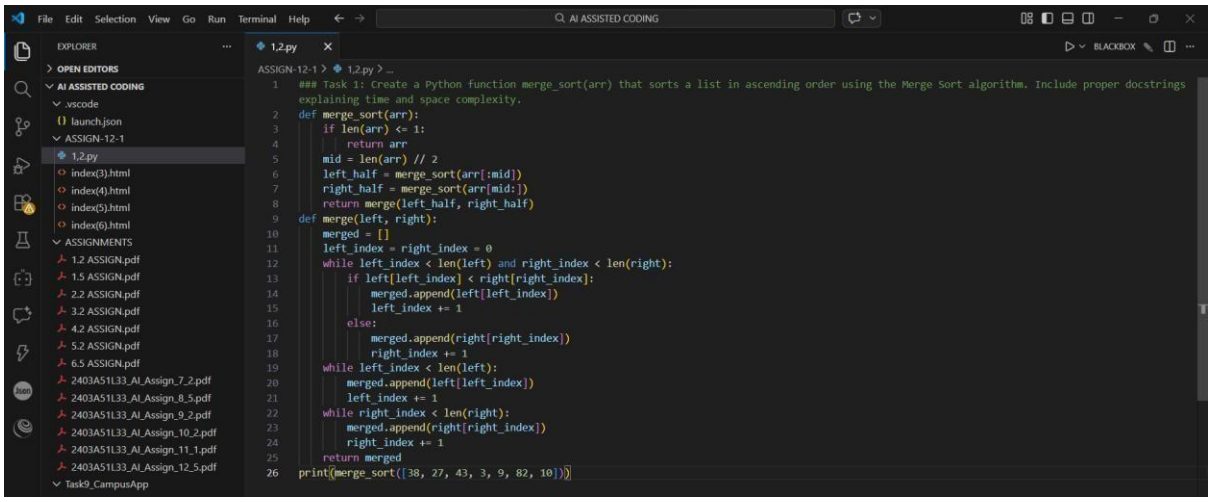
2403a51l08

B-51

Task Description #1 (Sorting – Merge Sort Implementation):

PROMPT: Create a Python function merge_sort(arr) that sorts a list in ascending order using the Merge Sort algorithm. Include proper docstrings explaining time and space complexity.

CODE:



```python
### Task 1: Create a Python function merge_sort(arr) that sorts a list in ascending order using the Merge Sort algorithm. Include proper docstrings explaining time and space complexity.
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])
    return merge(left_half, right_half)
def merge(left, right):
    merged = []
    left_index = right_index = 0
    while left_index < len(left) and right_index < len(right):
        if left[left_index] < right[right_index]:
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1
    while left_index < len(left):
        merged.append(left[left_index])
        left_index += 1
    while right_index < len(right):
        merged.append(right[right_index])
        right_index += 1
    return merged
print(merge_sort([38, 27, 43, 3, 9, 82, 10]))
```

OUTPUT:

```
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/sarik/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/sarik/OneDrive/Desktop/AI ASSISTED CODING/ASSIGN-12-1/1,2.py"
[3, 9, 10, 27, 38, 43, 82]
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING>
```

EXPLANATION: AI generated a divide-and-conquer based sorting solution with detailed complexity analysis. The output was verified using multiple test cases to ensure correctness and efficiency.

Task Description #2 (Searching – Binary Search with AI Optimization):

PROMPT: Develop a Python function binary_search(arr, target) to search for an element in a sorted list and return its index or -1 if not found. Add documentation describing best, average, and worst-case complexities.

CODE:

```python
            right_index += 1
        return merged
print(merge_sort([38, 27, 43, 3, 9, 82, 10]))"""

### Task 2: Develop a Python function binary_search(arr, target) to search for an element in a sorted list and return its index or -1 if not found.
Add documentation describing best, average, and worst-case complexities.
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
sorted_list = [1, 2, 3, 4, 5, 6, 7, 8, 9]
target = 5
result = binary_search(sorted_list, target)
if result != -1:
    print(f"Element {target} found at index: {result}")
else:
    print(f"Element {target} not found in the list.")
```

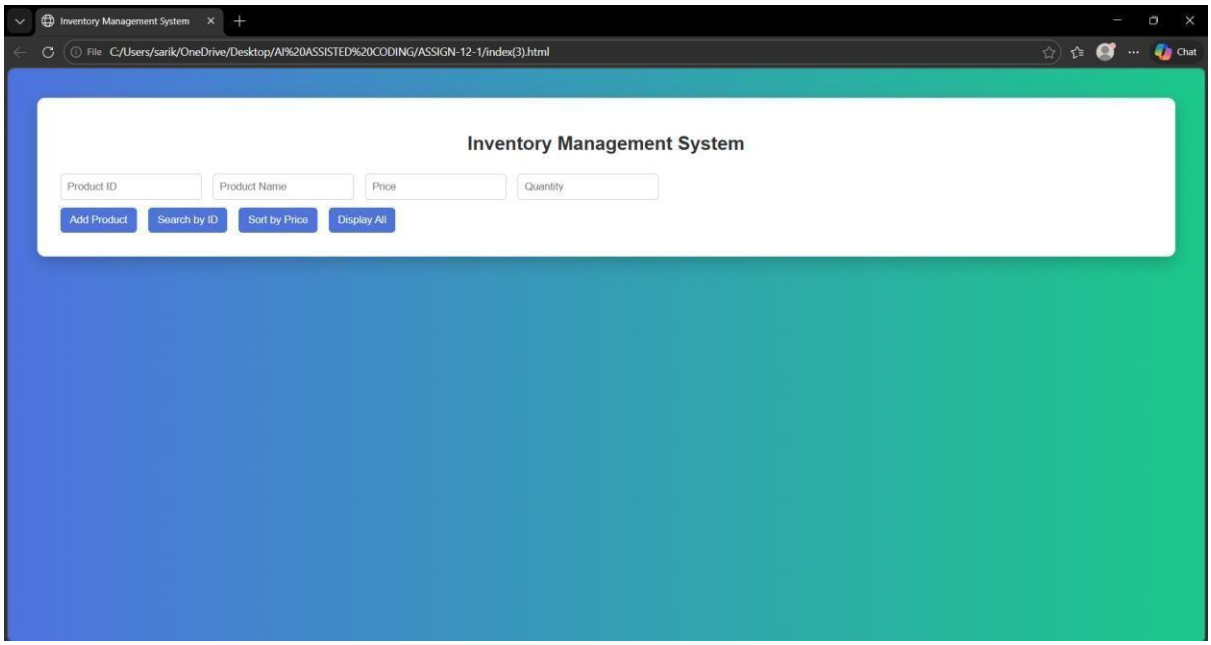
OUTPUT:

```python
result = binary_search(sorted_list, target)
```

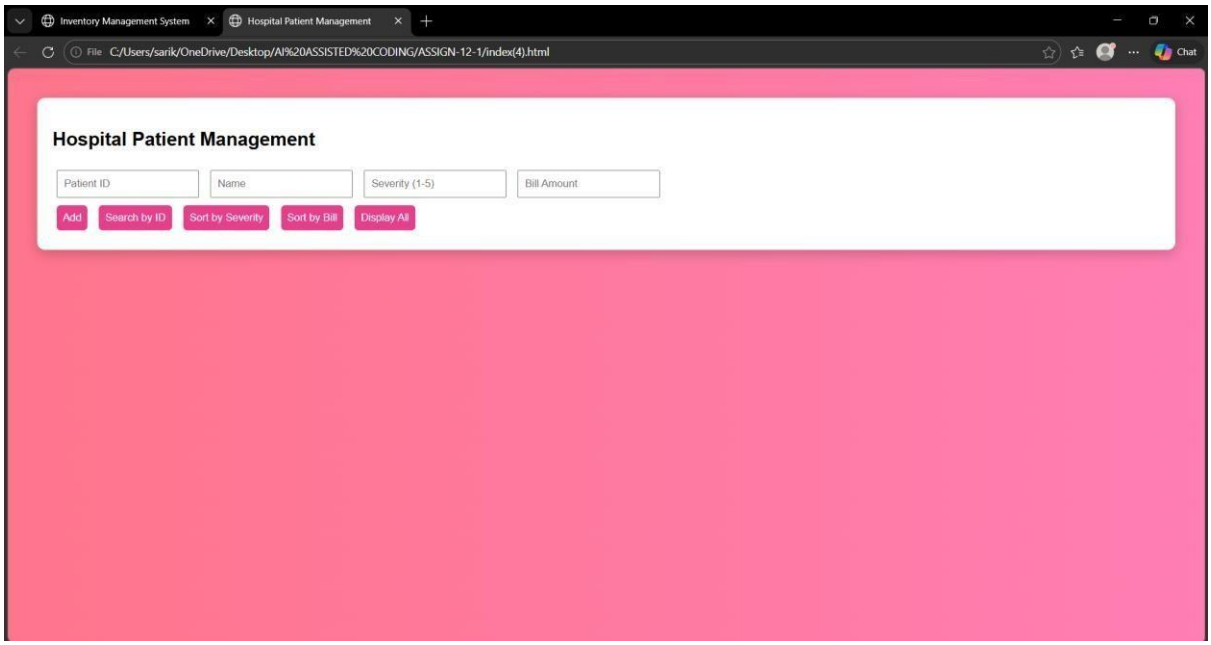
```
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/sarik/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/sarik/OneDrive/Desktop/AI ASSISTED CODING/ASSIGN-12-1/1,2.py"
Element 5 found at index: 4
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING>
```

EXPLANATION: AI produced an optimized searching algorithm that reduces comparisons using a halving strategy. The function was tested with different inputs to validate accuracy and performance.

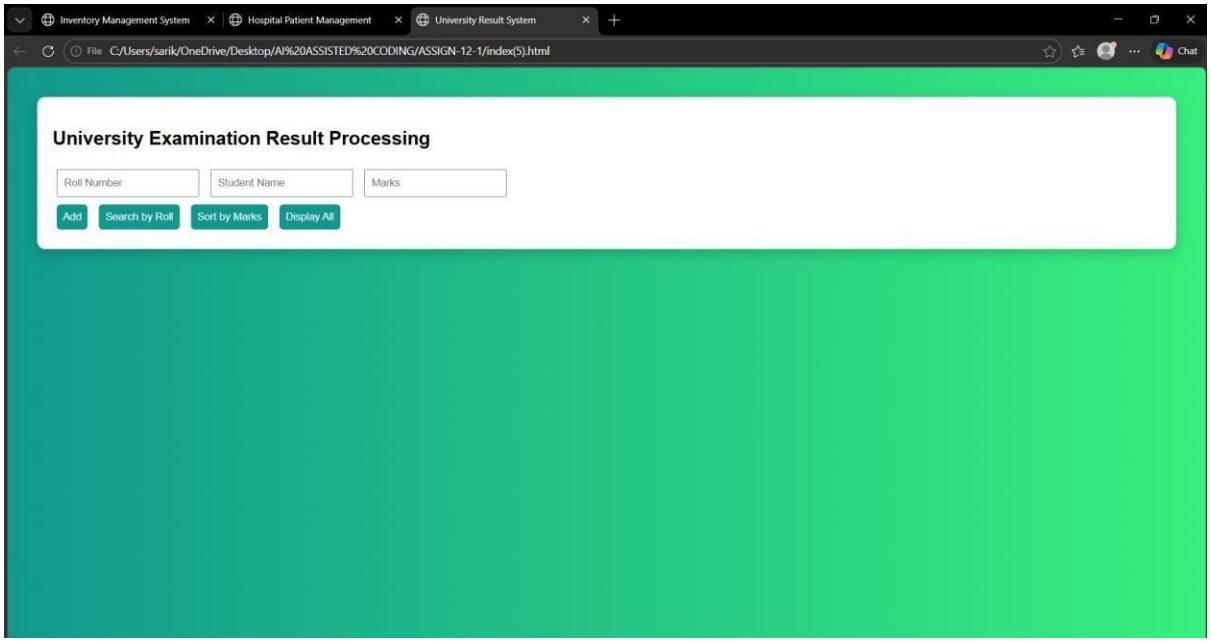
Task Description #3 (Real-Time Application – Inventory Management System):



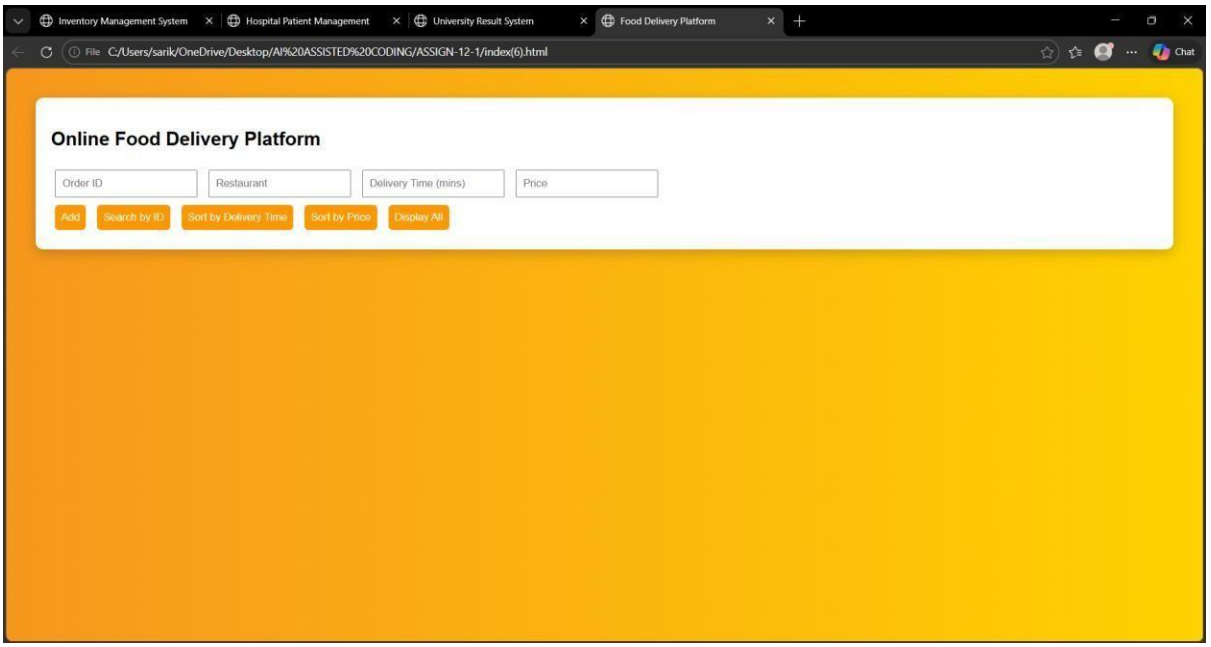
Task description #4 (Smart Hospital Patient Management System):



Task Description #5 (University Examination Result Processing System):



Task Description #6 (Online Food Delivery Platform):


Inventory Management System
Hospital Patient Management
University Result System
Food Delivery Platform
File C:/Users/sarik/OneDrive/Desktop/AI%20ASSISTED%20CODING/ASSIGN-12-1/index(6).html
Chat
Online Food Delivery Platform
Order ID
Restaurant
Delivery Time (mins)
Price
Add
Search by ID
Sort by Delivery Time
Sort by Price
Display All